Creating a Table with a Primary Key

Lecture Notes · Game DB Junior

In this lesson

- How to make a **users** table with an **id** column
- The **SERIAL** data type: Postgres makes the id for us
- Marking a column as **PRIMARY KEY**

1. The goal

Enough lecturing. Now we write SQL to build a relationship between two tables. First we make a **users** table with two columns:

- **id** — set up as a **primary key** column. Postgres treats it in a very special way.
- **username** — VARCHAR(50) for now.

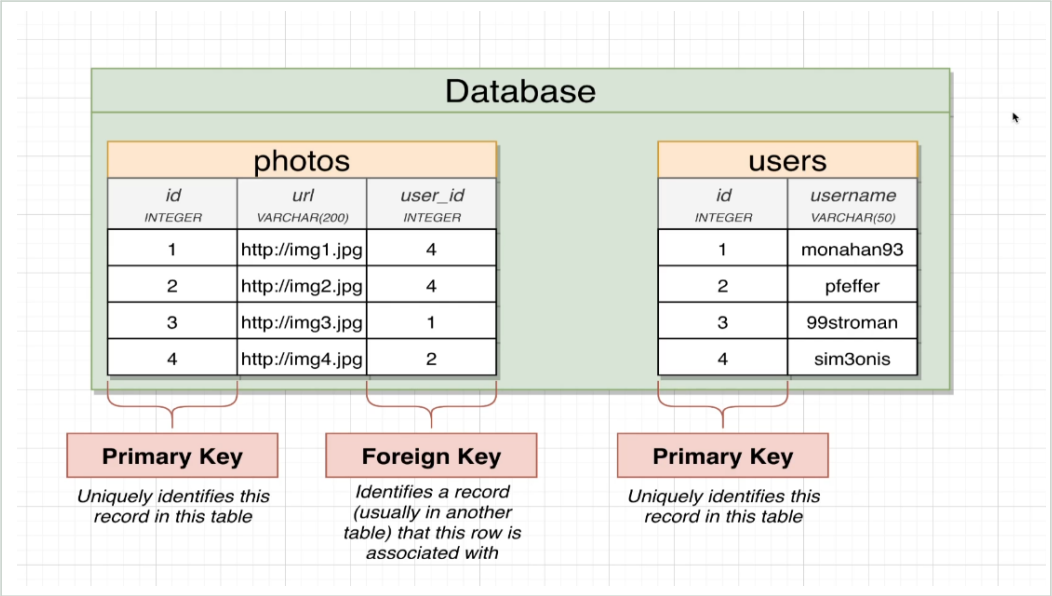


Figure 1 — The photos and users tables we want to build

Optional: you can **reset your database** to start with a clean slate. This removes all the tables in it (like the old cities table).

2. The CREATE TABLE statement

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50)
);
```

SERIAL

- The diagram shows the id type as INTEGER. But if we only used INTEGER, **we** would have to make all the ids, and write code to keep them unique.
- Instead, we use the data type **SERIAL**. It tells Postgres to **make the values for us automatically**.
- When we insert a user, we do **not** give an id. We only give a username. Postgres makes the id.
- It starts at **1**. For each new user, it takes the last id and **adds one**: 1, 2, 3, 4 ...

PRIMARY KEY

- After SERIAL, we mark the column as **PRIMARY KEY**.
- This gives special **performance benefits** when we look up records in users by their id.

username VARCHAR(50)

- We cannot insert a string longer than 50 characters. If we do, we get an error.

3. Inserting users

The id is made automatically, so we only list the **username** column. We add four users in one statement:

```
INSERT INTO users (username)
VALUES
    ('monahan93'),
    ('pfeffer'),
    ('99stroman'),
    ('sim3onis');
```

- Put a **comma** after each set of parentheses. The last one gets a **semicolon** instead.
- Then **SELECT * FROM users;** shows the ids 1, 2, 3, 4 for the four users.
- Each id is 100% unique, and we do not plan to change the ids by hand. Id 1 is always monahan93, id 2 is always pfeffer, and so on.

4. Running the code (my practice)

Step 1: create the users table. I did not reset my database, so the cities table is still there.

```sql
-- Create a table to store users
CREATE TABLE users (
    -- SERIAL: Postgres makes the ID by itself (1, 2, 3, ...)
    -- PRIMARY KEY: each ID is unique and finding rows by ID is fast
    id SERIAL PRIMARY KEY,
    -- VARCHAR(50): text up to 50 characters; longer text gives an error
    username VARCHAR(50)
);
```

Figure 2 — CREATE TABLE users with comments

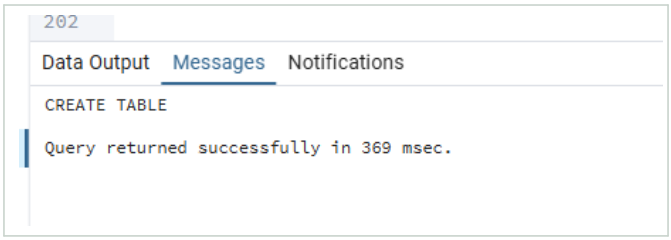


Figure 3 — CREATE TABLE ran successfully

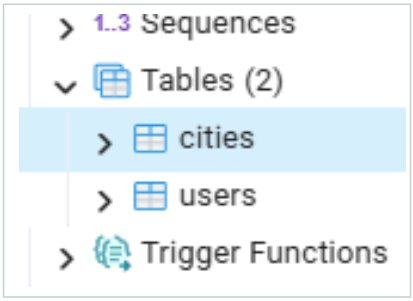


Figure 4 — Now there are 2 tables: cities and users

Step 2: insert 4 users. I only gave the username. The Messages tab shows **INSERT 0 4**.

```sql
-- Add 4 new users to the users table
-- We only give the username; Postgres makes the id by itself (SERIAL)
INSERT INTO users (username)
VALUES
    ('monohan93'),    -- gets id 1
    ('pferrer'),      -- gets id 2
    ('si93onis'),     -- gets id 3
    ('99stroman');    -- gets id 4 (last row ends with ; not ,)
```

Figure 5 — INSERT INTO users (username)



Figure 6 — INSERT 0 4

Step 3: SELECT * FROM users. Postgres made the ids 1, 2, 3, 4 by itself. The id column is marked **[PK] integer**.

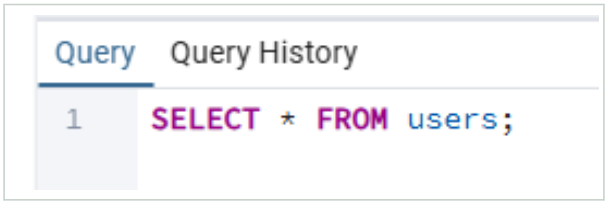


*Figure 7 — SELECT * FROM users*

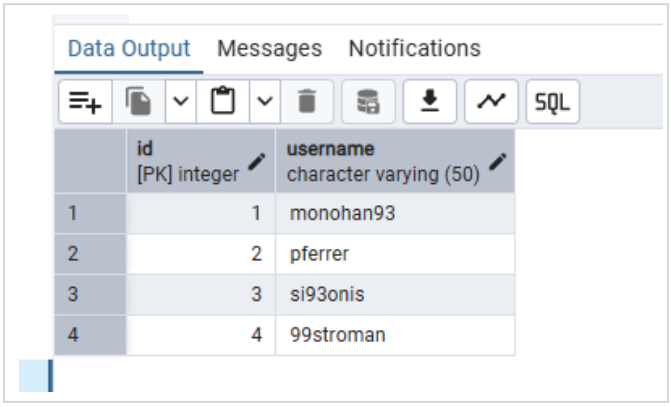


Figure 8 — ids 1 to 4 were made automatically

Note: my order and spelling are a bit different from the lecture. In the lecture, id 3 is 99stroman and id 4 is sim3onis. In my table, id 3 is si93onis and id 4 is 99stroman (and pfeffer is written pferrer). The ids follow the order of the INSERT.

5. Summary

Idea	Remember
SERIAL	Postgres makes the id for us: 1, then +1 for each new row
PRIMARY KEY	Marks the id as the primary key. Faster lookups by id
INSERT without id	Only give the other columns. The id is made automatically
ids never change	Each id is unique and always points to the same user

Next lesson: the photos table, with a user_id column as a foreign key.

Sources: Lecture transcript and lecture image (Figure 1); my own practice screenshots (Figures 2–8)